

Relatório de Auditoria de Segurança — atelie-erp

Auditoria de segurança de aplicação — cinco categorias

Escopo auditado. Repositório **atelie-erp(1)** completo — backend (src/server, src/pages/api), frontend (src/ui, src/state, src/main.ts), infraestrutura (wrangler.toml, .dev.vars.example, waha/docker-compose.waha.yml, .github/workflows/ci.yml), migrations (0001–0019) e histórico git. Auditoria estática com verificação de exequibilidade dos achados críticos.

Resultado. 11 achados: 2 críticos, 3 altos, 3 médios, 1 baixos e 2 informativos; mais 8 controles verificados como corretos.

Nota metodológica — mapeamento por stack

A stack detectada é **Astro 7 SSR sobre Cloudflare Workers**, com **Cloudflare D1 (SQLite)** e **SQL cru — sem ORM**, autenticação por **sessão própria** (tabela *sessions*, token Bearer, senhas PBKDF2-SHA256) e frontend **SPA imperativa em DOM** (TypeScript puro, sem React/Vue). Não há Supabase, portanto **RLS** foi mapeado para o equivalente desta stack: filtro explícito por usuário/tenant em cada query e verificação de posse em cada handler por id. Não existe *v-html/dangerouslySetInnerHTML*; o equivalente é **innerHTML com template literals**, auditado sink a sink. O deploy é feito por **wrangler** (não há Docker/Helm/Terraform no deploy principal; existe um docker-compose apenas para o motor WAHA em desenvolvimento).

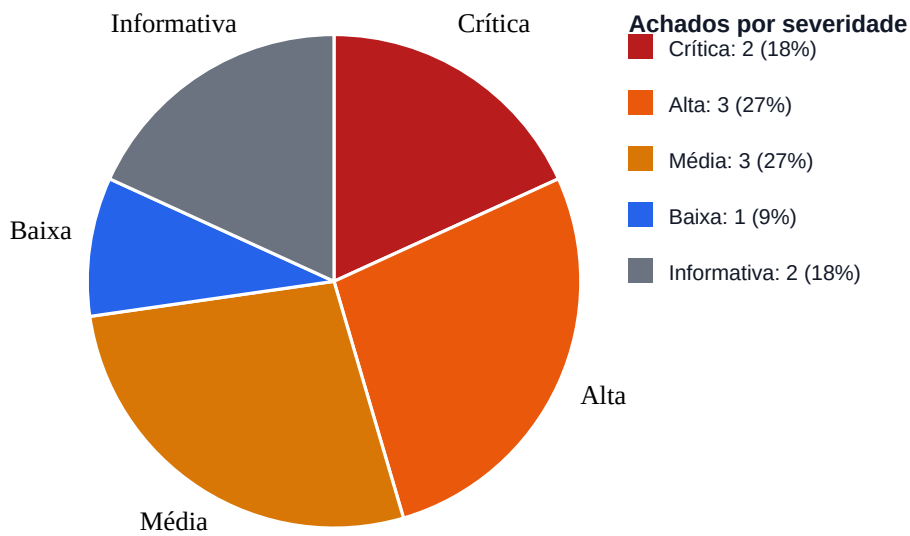
Camada	Tecnologia detectada	O que foi auditado
Linguagem	TypeScript (strict)	todo o src/
Framework	Astro 7 SSR / @astrojs/cloudflare	rotas em src/pages/api/**
Persistência	Cloudflare D1 (SQLite), SQL cru, sem ORM	src/server/sql.ts, crud.ts, mapping.ts
Auth	Sessão própria (Bearer + tabela sessions)	src/middleware.ts, src/server/auth.ts
Autorização	AUSENTE — src/server/authz.ts vazio (0 bytes)	categoria 2
Frontend	SPA imperativa (DOM + innerHTML), sem framework	src/ui/**, src/main.ts
Deploy/CI	wrangler.toml, GitHub Actions, docker-compose (WAHA dev)	seção de segredos

1. Resumo executivo

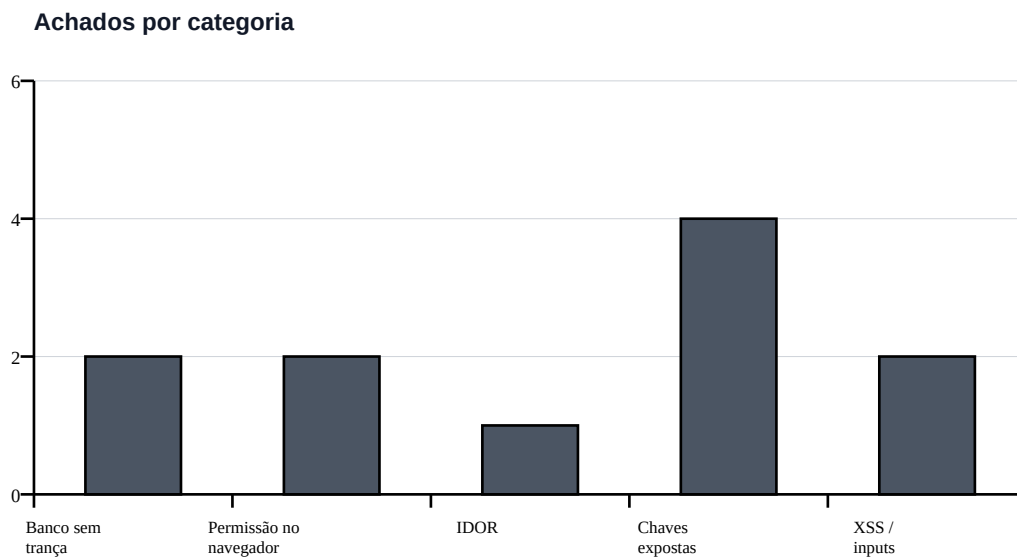
A aplicação implementa autenticação de forma consistente, mas **não implementa autorização**: `src/server/authz.ts` existe com 0 bytes e nenhum handler compara o papel de quem chama. Duas falhas críticas foram confirmadas: **SQL injection** por nome de coluna em `src/server/sql.ts:10-14,22-25`, alcançável por qualquer rota de escrita, e **escalada de privilégio total** em `/api/users`, onde um viewer cria admin, altera papéis e redefine a senha de qualquer conta. A varredura das 20 rotas por id mostrou que apenas `users/[id].ts` cruza uma fronteira de privilégio real; as demais pertencem a recursos compartilhados da única organização.

Severidade	Qtd	Significado
Crítica	2	Comprometimento total do banco/da aplicação, sem pré-condição além de ter uma sessão.
Alta	3	Explorável por usuário autenticado ou anônimo, com impacto relevante (credencial, configuração, controle de serviço).
Média	3	Condicional à configuração/uso; requer hardening.
Baixa	1	Superfície de exposição ou prática defensiva.
Informativa	2	Contexto, cobertura de auditoria ou item confirmado correto.

Achados por severidade



Achados por categoria



2. Pontos fortes (verificados)

Controles efetivamente auditados e **corretos** — registrados para provar a cobertura da auditoria.

Controle	Evidência
Hash de senha por usuário	src/server/auth.ts: hash/verify PBKDF2-SHA256, 100k iterações, salt por usuário; sessions com token como PK (migration 0003_crm.sql).
Sessão verificada no middleware	src/middleware.ts: exige Bearer para tudo fora de PUBLIC_PATHS (linhas 17-24) antes de o handler rodar.
LGPD: escopo do titular correto	me/data.ts:54-62, me/export.ts:45-50, me/erase.ts:74,86,136 — filtram por assignedUserId/assigneeUserId ao responder sobre o próprio titular.
CSP restritiva e sem frame-ancestors	src/middleware.ts:26-29 — default-src 'self', script-src 'self' (sem unsafe-inline), frame-ancestors 'none', base-uri 'self'.
Rate limit em rotas de sessão	src/middleware.ts:88-94, limitado via LIMITED_PATHS antes de qualquer leitura de credencial.
Sem SQL de leitura interpolado por valor	src/server/sql.ts: os VALUES do INSERT são placeholders parametrizados (linha 13); a injeção auditada afeta apenas o nome da coluna.
Mensagens do WhatsApp escapadas em texto	src/ui/views/crm/CrmInboxView.ts:710 — escape de texto no sink de texto, que é o uso correto da função.
Histórico git sem segredo real	git log --all -p: apenas placeholders de WAHA_API_KEY (prefixos e2e-pl e x), zero chaves privadas, zero tokens de provedor.

3. Pontos fracos (riscos centrais)

- **Autorização inexistente.** src/server/authz.ts tem 0 bytes. Nenhuma rota valida papel; toda a restrição de privilégio está na UI.
- **CRUD genérico sem filtro de coluna nem de posse.** O mesmo routeFactory serve dozens de endpoints, propagando SQL injection e IDOR em série.
- **Endpoint de dados LGPD copia o banco inteiro** para qualquer usuário autenticado, com mensagens e clientes sem recorte.
- **Segredos de produção dependem de configuração opcional** (webhook fail-open, credencial padrão do admin no seed).

4. Achados detalhados por categoria

4.1 1 · Banco sem trança

Severidade	Arquivo:linha	Descrição
Crítica	src/server/sql.ts:10-14, 22-25	SQL injection: nomes de coluna interpolados no SQL buildInsert e buildUpdate montam a lista de colunas a partir de Object.keys(row) e interpolam esse texto direto no SQL, sem validar contra as colunas reais da tabela. mapping.ts:33 repassa todas as chaves recebidas no body, sem whitelist.
Informativa	src/pages/api/me/data.ts:54-62; me/export.ts:45-50; me/erase.ts:74,86,136	Sem isolamento por linha — e assignedUserId NÃO é fronteira A aplicação é single-tenant por decisão declarada (README: 'sem multi-tenant'). Os campos assignedUserId/assigneeUserId são rótulos de responsável comercial e são usados apenas para delimitar o escopo LGPD do próprio titular. Não existe filtro por dono em nenhuma listagem, em nenhuma tabela.

4.2 2 · Permissão no navegador

Severidade	Arquivo:linha	Descrição
Crítica	src/pages/api/users/index.ts:11,52; users/[id].ts:23-45, 47-56, 58-67, 73	Escalada de privilégio: qualquer autenticado gerencia usuários As rotas de equipe não verificam o papel de quem chama. POST aceita qualquer valor de ROLES para o novo usuário (index.ts:11 e 52). PUT altera `role` de qualquer usuário (users/[id].ts:73) e redefine a senha de qualquer usuário (:58-67). DELETE remove qualquer usuário (:47-56). Nenhum handler compara user.role com qualquer coisa.
Média	src/pages/api/settings.ts	Configurações do negócio sem verificação de papel A rota de configurações é exigida pelo middleware (não está em PUBLIC_PATHS) e portanto exige sessão válida, mas não verifica papel: qualquer usuário autenticado, inclusive viewer, altera as configurações do ERP.

4.3 3 · IDOR (varredura exaustiva)

Severidade	Arquivo:linha	Descrição
Informativa	src/pages/api/**/[id].ts (20 rotas)	Varredura exaustiva dos 20 handlers [id].ts As 20 rotas por id foram enumeradas e lidas: components, crm/activities, appointment-types, calendar-events, catalog-products, contacts, conversation-notes, conversations, deals, messages, pipelines, quick-replies, stages, tags, tasks, customers, ingredients, orders, products, users. routeFactory.ts:59-79 aplica PUT/DELETE por context.params.id sem teste de posse.

4.4 4 · Chaves expostas

Severidade	Arquivo:linha	Descrição
Alta	waha/.env (WAHA_API_KEY_SHA512)	waha/.env versionado no git contém o verificador da credencial O arquivo waha/.env está rastreado pelo git (git ls-files confirma) e contém WAHA_API_KEY_SHA512, o hash SHA-512 que o docker-compose usa como credencial padrão quando o arquivo não existe. O .gitignore protege apenas .dev.vars (linha 5) e não tem nenhuma regra para .env.
Alta	migrations/0004_crm_seed.sql:5-9; README.md:73	Credencial padrão do admin sem troca obrigatória O seed cria seed-user-admin / admin@deskcomm.local com senha 'admin123' e salt fixo 'deskcomm-seed-v1'. O README documenta o par no texto e ensina a rodar `npm run db:seed:remote` contra produção.
Alta	src/middleware.ts:17-24 (PUBLIC_PATHS); src/pages/api/w hatsapp/session.ts:45	GET /api/whatsapp/session expõe QR de pareamento sem autenticação A rota está no conjunto PUBLIC_PATHS e o handler GET não faz nenhuma verificação de token (não há userFromToken no arquivo). O comentário em middleware.ts:14-16 justifica a exposição para permitir o pareamento antes do login.
Média	src/middleware.ts:17-24 (PUBLIC_PATHS); src/pages/api/w hatsapp/session.ts:84	DELETE /api/whatsapp/session destrói a sessão sem autenticação A rota inteira está fora da autenticação do middleware e o handler DELETE também não verifica token. O comentário em session.ts:162 descreve a deleção do engine session (auth incluso) como reset.

4.5 5 · XSS / inputs

Severidade	Arquivo:linha	Descrição
Média	src/domain/format.ts:33-37; src/ui/vi ews/crm/crmUi.ts:57,76,82,92; src/ui/views/CustomersView.ts:117	escapeHtml não codifica aspas: injeção de atributo HTML escapeHtml faz textContent -> innerHTML num div e devolve o HTML resultante. O truque neutraliza &, < e >, mas NÃO " nem '. Vários sinks interpolam o resultado em atributo com aspas duplas (value="...", title="..."), então uma aspa no dado fecha o atributo e abre marcação.

Severidade	Arquivo:linha	Descrição
Baixa	src/ui/views/crm/CrmInboxView.ts:7 10	waStatus interpolado em atributo Estado vindo do servidor entra em atributo HTML através do mesmo escape que não codifica aspas.

5. Detalhamento técnico e exploração

Para cada achado: CWE, caminho de ataque, impacto, correção sugerida e critérios de aceite verificáveis.

1. SQL injection: nomes de coluna interpolados no SQL **Crítica**

Arquivo:linha: src/server/sql.ts:10-14, 22-25 CWE: CWE-89

Por que é explorável. Um PUT/POST com uma chave de objeto que é uma expressão SQL (por exemplo `nome = (SELECT ...) WHERE 1=1 --`) entra na lista de colunas e vira SQL executável. Confirmado por prova local: a chave ■■■ na cláusula SET e a contagem de placeholders ficou igual à de valores.

Impacto. Um usuário autenticado consegue alterar dados de qualquer tabela exposta por rota CRUD e extrair conteúdo via expressão SQL. Afeta todas as rotas de escrita (POST/PUT).

Sugestão de correção. Validar as chaves recebidas contra a lista real de colunas da tabela (allowlist derivada de tables.ts) e descartar chaves desconhecidas antes de montar o SQL.

Critérios de aceite.

- Chave fora da allowlist é ignorada, não interpolada
- Teste com payload em nome de propriedade não altera schema nem executa subquery
- Teste cobrindo buildInsert e buildUpdate

2. Sem isolamento por linha — e assignedUserId NÃO é fronteira **Informativa**

Arquivo:linha: src/pages/api/me/data.ts:54-62; me/export.ts:45-50; me/erase.ts:74,86,136 CWE: CWE-284

Por que é explorável. Não explorável como vazamento entre tenants: não há tenant. Como todo usuário autenticado enxerga a base inteira por desenho, não existe fronteira de posse a ser atravessada.

Impacto. Qualquer conta autenticada — inclusive viewer — lê e grava todos os registros. Isso é coerente com o produto, mas precisa estar explícito: assignedUserId não deve ser tratado como controle de acesso em lugar nenhum.

Sugestão de correção. Documentar no README que o acesso é global por papel e que assignedUserId é metadado de atribuição, não autorização. Se isolamento por responsável for desejado, isso é nova feature com migration e policy explícita — não um bug.

Critérios de aceite.

- README declara que não há isolamento por linha
- Nenhum código trata assignedUserId como autorização
- Policy de acesso por papel documentada para rotas privilegiadas

3. Escalada de privilégio: qualquer autenticado gerencia usuários **Crítica**

Arquivo:linha: src/pages/api/users/index.ts:11,52; users/[id].ts:23-45, 47-56, 58-67, 73 CWE: CWE-269 / CWE-862

Por que é explorável. Autentique-se como viewer (papel padrão do schema, migrations/0003_crm.sql:11). Faça POST /api/users com role:'admin' e obtenha uma conta administrativa; em seguida PUT /api/users/<alvo> com password redefine a senha de qualquer conta, inclusive da administrativa. O frontend esconde o botão de exclusão para não-admin (CrmEquipeView.ts:102-104), mas a chamada direta funciona.

Impacto. Tomada de conta completa e criação de administrador por qualquer usuário autenticado. É a falha mais grave do sistema, porque anula toda a separação de papéis.

Sugestão de correção. Criar requireRole(...) em src/server/authz.ts (hoje o arquivo existe com 0 bytes) e aplicar nas rotas de usuários: só admin/manager listam, criam e editam; só admin deleta; alteração de papel e reset de senha restritas a admin.

Critérios de aceite.

- POST /api/users com viewer retorna 403
- PUT /api/users/<id> com role por viewer retorna 403
- PUT /api/users/<id> com password por não-admin retorna 403
- DELETE /api/users/<id> por manager retorna 403
- Testes cobrem as quatro rotas × viewer/agent/manager/admin

4. Configurações do negócio sem verificação de papel **Média**

Arquivo:linha: src/pages/api/settings.ts CWE: CWE-862

Por que é explorável. Com um token de viewer válido, um PUT/POST em /api/settings é aceito. O middleware em src/middleware.ts garante apenas autenticação; não há decisão de autorização por papel em nenhuma rota.

Impacto. viewer consegue alterar parâmetros que os demais usuários dependem (margens, nomes do negócio).

Sugestão de correção. Exigir admin (ou manager) em /api/settings via requireRole, alinhado à política de acesso por papel.

Critérios de aceite.

- PUT /api/settings com viewer retorna 403
- Somente admin/manager obtêm 200

5. Varredura exaustiva dos 20 handlers [id].ts **Informativa**

Arquivo:linha: src/pages/api/**/[id].ts (20 rotas) CWE: CWE-639

Por que é explorável. 19 das 20 rotas atacam recursos compartilhados da única organização (contatos, pedidos, produtos, funil). Como o acesso é global por papel desde a origem, buscar um id de outro usuário não cruza nenhuma fronteira de autorização: não há IDOR a explorar nesses recursos. A única rota que cruza uma fronteira real é users/[id].ts, tratada como escalada de privilégio em Categoria 2.

Impacto. Nenhum impacto de IDOR além do já reportado na Categoria 2. Registrado para deixar a varredura completa explícita.

Sugestão de correção. Nenhuma correção de IDOR é devida nos 19 recursos compartilhados. A correção de users/[id].ts é a da Categoria 2. Se isolamento por responsável entrar no escopo, todas as 20 rotas passam a precisar de verificação de posse.

Critérios de aceite.

- Varredura dos 20 handlers registrada no relatório
- Nenhuma rota por id está segura por medida de frontend
- users/[id].ts corrigido conforme Categoria 2

6. waha/.env versionado no git contém o verificador da credencial **Alta**

Arquivo:linha: waha/.env (WAHA_API_KEY_SHA512) CWE: CWE-798 / CWE-540

Por que é explorável. O próprio arquivo documenta a rotação: o valor é derivado do plaintext da WAHA_API_KEY. Qualquer pessoa com leitura do repositório ou de um clone recebe o material de autenticação do motor WAHA e a URL do webhook.

Impacto. Credencial do motor de WhatsApp exposta no histórico do repositório. Um clone do projeto já entrega esse valor.

Sugestão de correção. Remover waha/.env do controle de versão (git rm --cached), adicionar .env ao .gitignore, rotacionar WAHA_API_KEY e reescrever o histórico para remover o valor antigo.

Critérios de aceite.

- .env não aparece em git ls-files
- .gitignore contém regra .env
- WAHA_API_KEY rotacionada e o par plaintext/hash sincronizado
- Busca no histórico não retorna mais credencial

7. Credencial padrão do admin sem troca obrigatória **Alta**

Arquivo:linha: migrations/0004_crm_seed.sql:5-9; README.md:73 **CWE:** CWE-798 / CWE-1392

Por que é explorável. Login com admin@deskcomm.local / admin123 funciona em qualquer ambiente onde o seed foi aplicado. Combinado com a Categoria 2 (qualquer autenticado vira admin) e com a ausência de troca obrigatória, a conta do seed não tem nenhuma barreira restante.

Impacto. Acesso administrativo trivial a quem conhece a documentação pública do projeto.

Sugestão de correção. Exigir troca de senha no primeiro login (flag must_change_password no users, verificada pelo middleware), remover a senha do README e impedir db:seed:remote de criar admin interativo.

Critérios de aceite.

- Login com senha padrão é rejeitado até a troca
- README não documenta a senha do seed
- Seed remoto exige --allow-seed-admin explícito

8. GET /api/whatsapp/session expõe QR de pareamento sem autenticação **Alta**

Arquivo:linha: src/middleware.ts:17-24 (PUBLIC_PATHS); src/pages/api/whatsapp/session.ts:45 **CWE:** CWE-306

Por que é explorável. Qualquer requisição anônima a GET /api/whatsapp/session recebe o estado da sessão e o QR de pareamento do número. O QR é a credencial de controle do WhatsApp.

Impacto. Anônimos podem fotografar o QR e parear o número, ou usar a rota para sondar o estado da integração.

Sugestão de correção. Exigir sessão autenticada em GET/DELETE e expor o QR apenas a admin. Se o pareamento pré-login for realmente necessário, usar um token de uso único e curto, gerado por admin, em vez de deixar a rota aberta.

Critérios de aceite.

- GET /api/whatsapp/session sem token retorna 401
- QR só é devolvido para admin autenticado
- O comentário em middleware.ts reflete a decisão final

9. DELETE /api/whatsapp/session destrói a sessão sem autenticação **Média**

Arquivo:linha: src/middleware.ts:17-24 (PUBLIC_PATHS); src/pages/api/whatsapp/session.ts:84 **CWE:** CWE-306 / CWE-862

Por que é explorável. Um DELETE anônimo reinicia a integração de WhatsApp: a sessão pareada é destruída e o número precisa ser pareado de novo.

Impacto. DoS de integração: qualquer anônimo derruba o atendimento por WhatsApp repetidamente.

Sugestão de correção. Exigir admin autenticado em DELETE, e separar a rota pública de pareamento da rota destrutiva.

Critérios de aceite.

- DELETE sem token retorna 401
- Somente admin executa o reset

10. escapeHtml não codifica aspas: injeção de atributo HTML **Média**

Arquivo:linha: src/domain/format.ts:33-37; src/ui/views/crm/crmUi.ts:57,76,82,92; src/ui/views/CustomersView.ts:117 **CWE:** CWE-79

Por que é explorável. Um nome de contato contendo `"` seguido de tag quebra o atributo value e injeta elementos no DOM. Confirmado que escapeHtml não escapa aspas. A CSP do projeto (middleware.ts:26-29, script-src 'self') bloqueia handlers inline e javascript:, então a execução de script não é demonstrada; o impacto comprovado é injeção de HTML/DOM.

Impacto. Injeção de HTML no DOM com dado controlado por usuário: manipulação de formulário, clobbering e alteração do que a vítima vê. Sem bypass da CSP confirmado, não é XSS executável comprovado.

Sugestão de correção. Usar um escapeAtrib que também trate " e ' (ou escrever o valor via property value no DOM, em vez de interpolar em template string). Manter escapeHtml para contexto de texto.

Critérios de aceite.

- escapeAtrib escapa " e '
- Teste com aspas em valor de campo não cria elemento no DOM
- Busca por sinks em atributo passa a usar escapeAtrib

11. waStatus interpolado em atributo **Baixa**

Arquivo:linha: src/ui/views/crm/CrmInboxView.ts:710 **CWE:** CWE-79

Por que é explorável. Mesma classe do achado anterior, em um único sink e com dado derivado de status controlado pelo servidor.

Impacto. Superfície de injeção de atributo em um ponto isolado.

Sugestão de correção. Resolver junto do escapeAtrib, lendo o valor por property.

Critérios de aceite.

- Sink deixa de usar interpolação em atributo

6. Recomendações priorizadas

Prio.	Ação	Resolve
P0	Criar requireRole em src/server/authz.ts e fechar as rotas de usuários (POST, PUT, DELETE) e /api/settings.	Escalada de privilégio e Tomada de conta
P0	Remover waha/.env do git, adicionar .env ao .gitignore e rotacionar WAHA_API_KEY.	Credencial do motor exposta
P0	Exigir autenticação em /api/whatsapp/session, restringindo QR e reset a admin.	Controle do WhatsApp e DoS
P1	Validar chaves de coluna contra a allowlist de tables.ts antes de montar INSERT/UPDATE.	SQL injection em todas as escritas
P1	Exigir troca de senha no primeiro login e remover a senha padrão do README.	Acesso admin trivial
P2	Adotar escapeAtrib (ou property) em todo sink de atributo que recebe dado do usuário.	Injeção de HTML no DOM
P2	Documentar que não existe isolamento por linha e que assignedUserId é metadado, não autorização.	Risco de interpretação errada
P3	Revalidar periodicamente os controles listados como pontos fortes.	Evita regressão silenciosa

7. Issues para o GitHub

Texto completo de cada issue, pronto para copiar e colar. As mesmas questões estão em [docs/security-audit/issues-para-o-github.md](#).

ISSUE 1 — [Segurança] Qualquer usuário autenticado pode criar admin, alterar papéis e redefinir senhas

Labels sugeridas: security, critical, broken-access-control

Problema

As rotas de equipe não verificam o papel de quem chama. POST /api/users aceita qualquer valor de ROLES, PUT /api/users/[id] altera o campo `role` e redefine a senha de qualquer usuário, e DELETE remove qualquer conta. A UI esconde o botão de exclusão para não-admin, mas nenhuma rota consulta user.role. src/server/authz.ts existe com 0 bytes.

Evidência

```
src/pages/api/users/index.ts:11
const ROLES: Role[] = ['viewer', 'agent', 'manager', 'admin'];
src/pages/api/users/index.ts:52
role: body.role as User['role'],
src/pages/api/users/[id].ts:73
if (body.role !== undefined) patch.role = body.role as User['role'];
src/pages/api/users/[id].ts:58-67 (applyPassword: sem teste de papel)
src/pages/api/users/[id].ts:47-56 (DELETE: sem teste de papel)
src/ui/views/crm/CrmEquipeView.ts:102-104 (gate só no botão de excluir)
src/server/authz.ts (0 bytes)
```

Impacto

Um viewer autenticado cria conta admin, redefine a senha de um administrador e assume o controle. Anula toda a separação de papéis do produto.

Sugestão de correção

Implementar requireRole em src/server/authz.ts e aplicá-lo nas rotas de usuários e de settings.

Critérios de aceite

- POST /api/users com viewer retorna 403
- PUT /api/users/<id> com role ou password por não-admin retorna 403
- DELETE /api/users/<id> por manager retorna 403
- Cobertura de teste das quatro rotas × quatro papéis

ISSUE 2 — [Segurança] SQL injection por nome de coluna em INSERT/UPDATE

Labels sugeridas: security, critical, injection

Problema

buildInsert e buildUpdate interpolam Object.keys(row) diretamente no texto SQL sem validar contra as colunas reais. mapping.ts repassa todas as chaves do body, sem whitelist, de modo que uma chave maliciosa vira SQL executável em qualquer rota de escrita.

Evidência

```
src/server/sql.ts:10-14
const columns = Object.keys(row);
const cols = columns.join(', ');
const sql = `INSERT INTO ${table} (${cols}) VALUES (${placeholders})`;
src/server/sql.ts:22-25
const columns = Object.keys(row);
const setClause = columns.map((c) => `${c} = ?`).join(', ');
const sql = `UPDATE ${table} SET ${setClause} WHERE id = ?`;
src/server/mapping.ts:33
const out: Record<string, unknown> = { ...entity }; // sem whitelist
```

Impacto

Gravação arbitrária em qualquer tabela alcançável por uma rota CRUD e extração de dados por subquery.

Sugestão de correção

Derivar a allowlist de colunas de tables.ts, descartar chaves desconhecidas e montar o SQL apenas com colunas validadas.

Critérios de aceite

- Chave não permitida é descartada antes do SQL
- Payload em nome de propriedade não executa subquery
- Testes para buildInsert e buildUpdate

ISSUE 3 — [Segurança] waha/.env versionado carrega o verificador da credencial da WAHA

Labels sugeridas: security, high, secrets

Problema

waha/.env está rastreado pelo git e contém WAHA_API_KEY_SHA512, o hash que o docker-compose usa como credencial padrão. O .gitignore protege .dev.vars, mas não tem regra para .env.

Evidência

```
git ls-files waha/.env -> waha/.env
waha/.env
WAHA_API_KEY_SHA512=sha512:b6eb06b3...
WAHA_API_BASE_URL=...
.gitignore:5
.dev.vars # apenas este arquivo; sem regra .env
```

Impacto

Qualquer clone do repositório recebe o material de autenticação do motor de WhatsApp e a URL do webhook.

Sugestão de correção

git rm --cached waha/.env, adicionar .env ao .gitignore, rotacionar WAHA_API_KEY e reescrever o histórico.

Critérios de aceite

- .env ausente de git ls-files
- WAHA_API_KEY rotacionada e par plaintext/hash sincronizado
- Histórico sem o valor antigo

ISSUE 4 — [Segurança] /api/whatsapp/session sem autenticação expõe QR e permite reset destrutivo

Labels sugeridas: security, high, auth

Problema

A rota está no PUBLIC_PATHS do middleware e seus handlers GET e DELETE não verificam token. GET devolve o QR de pareamento; DELETE destrói a sessão pareada.

Evidência

```
src/middleware.ts:17-24
const PUBLIC_PATHS = new Set([... '/api/whatsapp/session']);
src/pages/api/whatsapp/session.ts:45 export const GET
src/pages/api/whatsapp/session.ts:84 export const DELETE
src/pages/api/whatsapp/session.ts:162 // reset destrutivo, sem auth
(nenhuma ocorrência de userFromToken no arquivo)
```

Impacto

Anônimo fotografa o QR e pareia o número, ou derruba a integração repetidamente, interrompendo o atendimento.

Sugestão de correção

Exigir admin autenticado em GET e DELETE; se o pareamento pré-login for necessário, usar token de uso único.

Critérios de aceite

- GET sem token retorna 401
- DELETE sem token retorna 401
- QR entregue apenas a admin

ISSUE 5 — [Segurança] escapeHtml não escapa aspas e permite injeção de atributo

Labels sugeridas: security, medium, xss

Problema

escapeHtml usa textContent -> innerHTML, o que escapa &, < e > mas preserva " e '. Vários sinks interpolam o retorno em atributo entre aspas duplas, permitindo fechamento do atributo e injeção de marcação.

Evidência

```
src/domain/format.ts:33-37
const div = document.createElement('div');
div.textContent = text ?? '';
return div.innerHTML; // " e ' NÃO são escapados
src/ui/views/crm/crmUi.ts:57,76,82,92 value="${...}"
src/ui/views/CustomersView.ts:117 value="${...}"
src/middleware.ts:26-29 script-src 'self' (sem unsafe-inline: bloqueia handler inline)
```

Impacto

Injeção de HTML/DOM com dado controlado por usuário (clobbering, manipulação de formulário, alteração do que a vítima vê). Execução de script não comprovada: a CSP bloqueia o caminho direto.

Sugestão de correção

Criar escapeAtrib tratando " e ', ou atribuir o valor por property no DOM em vez de interpolar em template string.

Critérios de aceite

- escapeAtrib escapa " e '
- Dado com aspa em campo não cria elemento no DOM
- Sinks de atributo migrados

ISSUE 6 — [Segurança] Credencial padrão do admin sem troca obrigatória

Labels sugeridas: security, high, secrets

Problema

O seed cria `admin@deskcomm.local / admin123` com salt fixo, o README documenta o par e ensina a rodar ``npm run db:seed:remote`` contra produção. Não há exigência de troca no primeiro login.

Evidência

```
migrations/0004_crm_seed.sql:5-9
-- Admin user — senha padrão "admin123"
('seed-user-admin', 'Administrador', 'admin@deskcomm.local',
 '022d504d...', 'admin', '2026-01-01T00:00:00.000Z');
README.md:73
Login padrão do CRM (criado pelo seed): `admin@deskcomm.local` / `admin123`.
```

Impacto

Acesso administrativo trivial a quem leia a documentação.

Sugestão de correção

Flag `must_change_password` verificada no middleware, remoção da senha do README e seed remoto exigindo flag explícita.

Critérios de aceite

- Login com senha padrão bloqueado até a troca
- README sem a senha do seed
- `db:seed:remote` exige `--allow-seed-admin`

ISSUE 7 — [Segurança] /api/settings sem verificação de papel

Labels sugeridas: security, medium, broken-access-control

Problema

A rota exige sessão válida (não está em `PUBLIC_PATHS`) mas não verifica papel, de modo que viewer altera configurações do negócio.

Evidência

```
src/middleware.ts:17-24 settings NÃO está em PUBLIC_PATHS
src/pages/api/settings.ts nenhuma verificação de role
```

Impacto

viewer altera parâmetros compartilhados (margens, nome do negócio).

Sugestão de correção

Exigir `admin/manager` via `requireRole`.

Critérios de aceite

- `PUT /api/settings` com viewer retorna 403

ISSUE 8 — [Docs] Registrar que não existe isolamento por linha e que assignedUserId não é autorização

Labels sugeridas: documentation, security

Problema

A aplicação é single-tenant e todas as listagens devolvem a base inteira. assignedUserId/assigneeUserId são rótulos de responsável, usados apenas no escopo LGPD do titular. Risco real: tratar esses campos como fronteira de acesso.

Evidência

```
src/pages/api/me/data.ts:54-62 uso em escopo LGPD
src/pages/api/me/export.ts:45-50 uso em escopo LGPD
src/pages/api/me/erase.ts:74,86,136 uso em escopo LGPD
src/server/routeFactory.ts:40-43 listagem sem WHERE de escopo
README.md 'sem multi-tenant'
```

Impacto

Nenhum contorno de segurança hoje (o modelo é global por papel), mas decisões futuras podem herdar uma suposição falsa de isolamento.

Sugestão de correção

Documentar no README a semântica de papel x acesso e o caráter não autorizativo de assignedUserId.

Critérios de aceite

- README descreve o modelo de acesso por papel
- Nenhum código usa assignedUserId como autorização

ISSUE 9 — [Segurança] Corrigir N+1 em me/data.ts e me/export.ts

Labels sugeridas: performance, medium

Problema

Ao montar o recorte LGPD, contacts é filtrado duas vezes e há carregamento sequencial de coleções, com uma chamada ao banco por coleção.

Evidência

```
src/pages/api/me/data.ts:54-62
all.contacts.filter(c => c.assignedUserId === user.id)
.map(c => c.id)
contacts: all.contacts.filter(c => c.assignedUserId === user.id),
// o mesmo contacts filtrado duas vezes
```

Impacto

Latência extra e trabalho duplicado no caminho mais sensível de privacidade.

Sugestão de correção

Filtrar uma vez em variável e reaproveitar; revisar o carregamento sequencial.

Critérios de aceite

- contacts é filtrado uma única vez por requisição
- Teste de LGPD garante o mesmo recorte após a mudança

ISSUE 10 — [Testes] Cobrir as rotas por id contra alteração de escopo

Labels sugeridas: testing, security

Problema

As 20 rotas [id].ts compartilham routeFactory.ts:59-79, que aplica PUT/DELETE por id sem teste de posse. Hoje não há teste travando esse contrato para as rotas privilegiadas.

Evidência

```
src/server/routeFactory.ts:59-79 (PUT/DELETE por params.id)
20 rotas por id em src/pages/api/**/[id[]].ts
testes ausentes para PUT/DELETE de users/[id].ts
```

Impacto

Uma correção de autorização aplicada só em um handler pode ser revertida sem o teste acusar.

Sugestão de correção

Criar testes parametrizados cobrindo as 20 rotas por id, com atenção especial a users/[id].ts.

Critérios de aceite

- Teste cobre PUT e DELETE de users/[id].ts por papel
- Matriz das 20 rotas por id presente